

Natural Language Processing 2

Building an LLM from Scratch —Attention Mechanism

Fatemeh Dehrouyeh

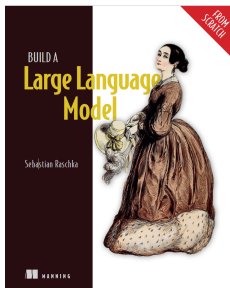


Contents

- 1 Introduction
- 2 A simple self-attention mechanism without trainable weights
- 3 Implementing self-attention with trainable weights
- 4 Causal Attention
- 5 Extending single-head attention to multi-head attention

Main Source

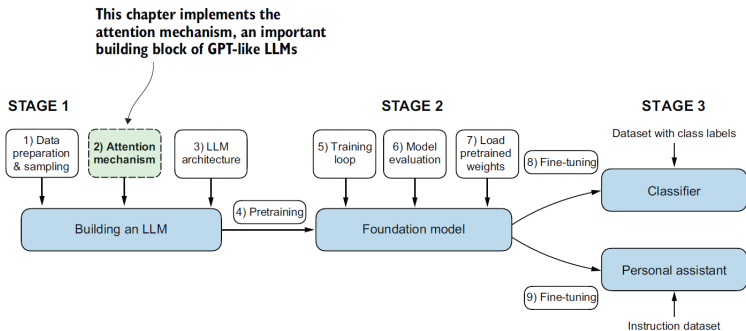
- **Video lectures:** YouTube lecture series
- **Source book:** *Build a Large Language Model (From Scratch)* by Sebastian Raschka
- **GitHub code:** <https://github.com/rasbt/LLMs-from-scratch>



Current Section

- 1 Introduction
- 2 A simple self-attention mechanism without trainable weights
- 3 Implementing self-attention with trainable weights
- 4 Causal Attention
- 5 Extending single-head attention to multi-head attention

Roadmap



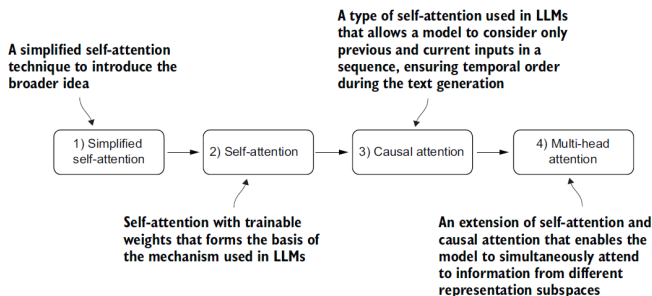
What Is Attention?

Attention is the mechanism that helps a model decide which words are most important when understanding or generating text.

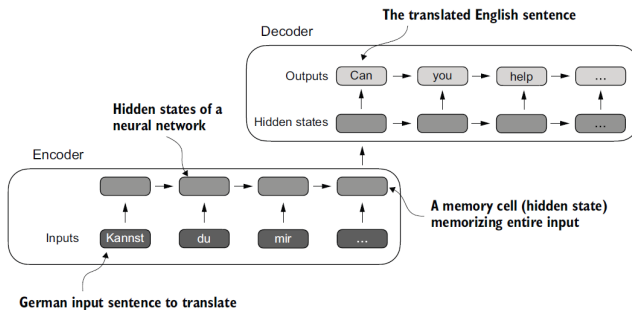
It is like the engine of a car: the Transformer is the full car, but attention is what powers it.

Different Attention Variants Built on Each Other

The figure depicts different attention mechanisms we will code in this chapter, starting with a simplified version of self-attention before adding the trainable weights. The causal attention mechanism adds a mask to self-attention that allows the LLM to generate one word at a time. Finally, multi-head attention organizes the attention mechanism into multiple heads, allowing the model to capture various aspects of the input data in parallel.

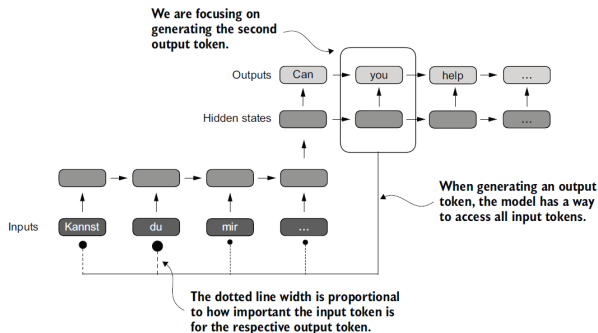


Encoder-Decoder RNNs



Before the advent of transformer models, encoder–decoder RNNs were a popular choice for machine translation. The encoder takes a sequence of tokens from the source language as input, where a hidden state (an intermediate neural network layer) of the encoder encodes a compressed representation of the entire input sequence. Then, the decoder uses its current hidden state to begin the translation, token by token.

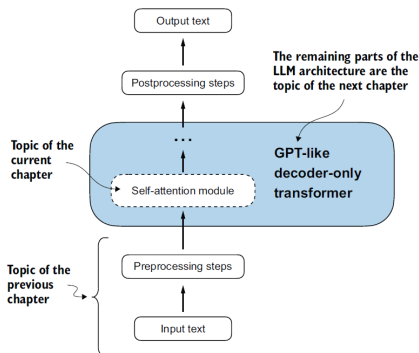
Attention Mechanism



Using an attention mechanism, the text-generating decoder part of the network can access all input tokens selectively. This means that some input tokens are more important than others for generating a given output token. The importance is determined by the attention weights, which we will compute later.

Self Attention

Self-attention is a mechanism in transformers used to compute more efficient input representations by allowing each position in a sequence to interact with and weigh the importance of all other positions within the same sequence.



Note: Scalars, Vectors, Matrices and Tensors

A scalar is just a single number.

2

Scalar

0D tensor

An example of a 3D vector that consists of 3 entries

$\begin{bmatrix} 3 \\ 1 \\ 3 \end{bmatrix}$

Vector

1D tensor

A matrix with 3 rows and 4 columns

$\begin{bmatrix} 3 & 5 & 1 & 2 \\ 1 & 7 & 2 & 3 \\ 3 & 3 & 4 & 9 \end{bmatrix}$

Matrix

2D tensor

Note: Creating PyTorch Tensors

```
import torch

tensor0d = torch.tensor(1)

tensor1d = torch.tensor([1, 2, 3])

tensor2d = torch.tensor([[1, 2],
                          [3, 4]])

tensor3d = torch.tensor([[[1, 2], [3, 4]],
                          [[5, 6], [7, 8]]])
```

Creates a zero-dimensional tensor (scalar) from a Python integer

Creates a one-dimensional tensor (vector) from a Python list

Creates a two-dimensional tensor from a nested Python list

Creates a three-dimensional tensor from a nested Python list

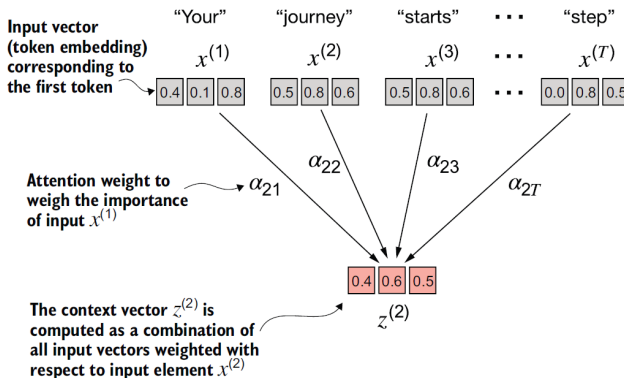
Current Section

- 1 Introduction
- 2 A simple self-attention mechanism without trainable weights**
- 3 Implementing self-attention with trainable weights
- 4 Causal Attention
- 5 Extending single-head attention to multi-head attention

Computing Context Vectors

- Suppose we are given an input sequence $x^{(1)}$ to $x^{(T)}$
 - The input is a text (for example, a sentence like “Your journey starts with one step”) that has already been converted into token embeddings
 - For instance, $x^{(1)}$ is a d -dimensional vector representing the word “Your”, and so forth
- **Goal:** compute context vectors $z^{(i)}$ for each input sequence element $x^{(i)}$ in $x^{(1)}$ to $x^{(T)}$ (where z and x have the same dimension)
 - A context vector $z^{(i)}$ is a weighted sum over the inputs $x^{(1)}$ to $x^{(T)}$
 - The context vector is “context”-specific to a certain input
 - Instead of $x^{(i)}$ as a placeholder for an arbitrary input token, let’s consider the second input, $x^{(2)}$
 - And to continue with a concrete example, instead of the placeholder $z^{(i)}$, we consider the second output context vector, $z^{(2)}$
 - The second context vector, $z^{(2)}$, is a weighted sum over all inputs $x^{(1)}$ to $x^{(T)}$ weighted with respect to the second input element, $x^{(2)}$
 - The attention weights are the weights that determine how much each of the input elements contributes to the weighted sum when computing $z^{(2)}$
 - In short, think of $z^{(2)}$ as a modified version of $x^{(2)}$ that also incorporates information about all other input elements that are relevant to a given task at hand

Computing Context Vectors—Visualization



This is called context vector because it has the information of all the inputs. Please note that the numbers in the figure are truncated to one digit after the decimal point.

The reference token, which is the second token here, is called the query.

Code Example

Suppose we have the following input sentence that is already embedded in 3-dimensional vectors as described in chapter 3 (we use a very small embedding dimension here for illustration purposes).

```
▶ import torch

inputs = torch.tensor(
    [[0.43, 0.15, 0.89], # Your      (x^1)
     [0.55, 0.87, 0.66], # journey (x^2)
     [0.57, 0.85, 0.64], # starts  (x^3)
     [0.22, 0.58, 0.33], # with    (x^4)
     [0.77, 0.25, 0.10], # one     (x^5)
     [0.05, 0.80, 0.55]] # step     (x^6)
)
```

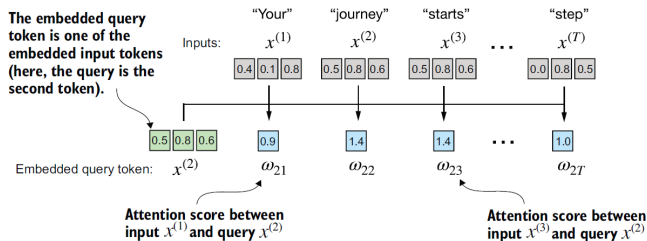

Step 1: Attention Scores

- **Step 1:** compute unnormalized attention scores ω
- Suppose we use the second input token as the query, that is, $q^{(2)} = x^{(2)}$, we compute the unnormalized attention scores via dot products:
 - $\omega_{21} = x^{(1)} q^{(2)\top}$
 - $\omega_{22} = x^{(2)} q^{(2)\top}$
 - $\omega_{23} = x^{(3)} q^{(2)\top}$
 - ...
 - $\omega_{2T} = x^{(T)} q^{(2)\top}$
- Above, ω is the Greek letter "omega" used to symbolize the unnormalized attention scores
 - The subscript "21" in ω_{21} means that input sequence element 2 was used as a query against input sequence element 1

By convention, the unnormalized attention weights are referred to as "attention scores" whereas the normalized attention scores, which sum to 1, are referred to as "attention weights".

Step 1 (Attention Scores)—Visualization

The goal is to illustrate how the context vector $z^{(2)}$ is computed using the second input element, $x^{(2)}$, as the query. This figure shows the first intermediate step: computing the attention scores ω between the query $x^{(2)}$ and all input elements using dot products.



The dot product between the query and each input vector gives us the attention score, which acts as a similarity metric.

Step1 (Attention Scores)—Code

```
▶ #Step 1: Attention Scores
query = inputs[1] # 2nd input token is the query

# Create an empty tensor with the same length as the number of input tokens to store their attention scores.
attn_scores_2 = torch.empty(inputs.shape[0])
for i, x_i in enumerate(inputs):
    attn_scores_2[i] = torch.dot(x_i, query) # dot product (transpose not necessary here since they are 1-dim vectors)

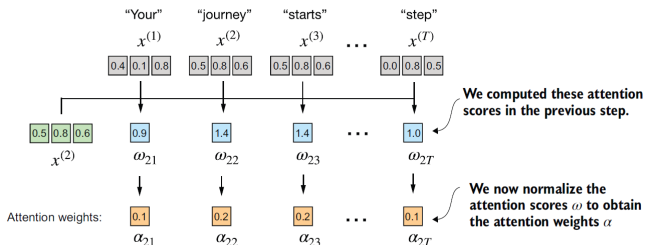
print(attn_scores_2)

... tensor([0.9544, 1.4950, 1.4754, 0.8434, 0.7070, 1.0865])
```

Step 2: Converting Attention Scores into Attention Weights

Step 2: normalize the unnormalized attention scores (“omegas”, ω) so that they sum up to 1

Here is a simple way to normalize the unnormalized attention scores to sum up to 1 (a convention, useful for interpretation, and important for training stability):



Step 2: Normalizing Using Softmax Function—Code

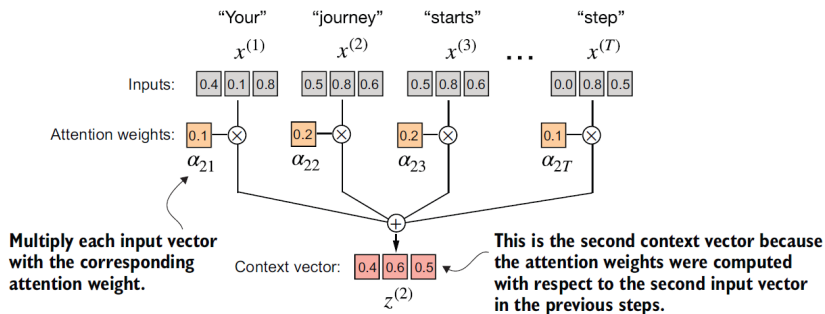
```
▶ #Step2: Attention Weights
attn_weights_2 = torch.softmax(attn_scores_2, dim=0)

print("Attention weights:", attn_weights_2)
print("Sum:", attn_weights_2.sum())

... Attention weights: tensor([0.1385, 0.2379, 0.2333, 0.1240, 0.1082, 0.1581])
Sum: tensor(1.)
```

Step3: Computing Context Vector

Step 3: compute the context vector $z^{(2)}$ by multiplying the embedded input tokens, $x^{(i)}$, with the attention weights and sum the resulting vectors:



A larger attention weight means that the corresponding input token contributes more to the context vector.

Step3 (Computing Context Vector)—Code



#Step3: Computing Context Vector

query = inputs[1] # 2nd input token is the query

context_vec_2 = torch.zeros(query.shape)

for i,x_i in enumerate(inputs):

 context_vec_2 += attn_weights_2[i]*x_i

print(context_vec_2)

... tensor([0.4419, 0.6515, 0.5683])

Generalization: Computing attention weights for all input tokens

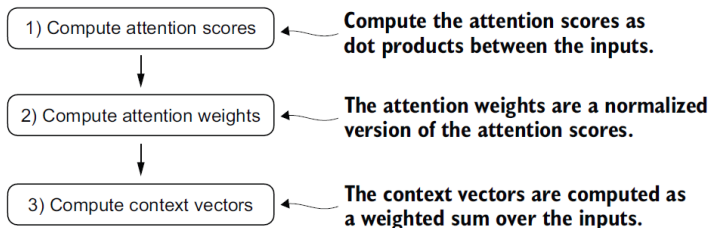
We computed the attention weights and context vector for input 2. Next, we are generalizing this computation to compute all attention weights and context vectors.

	Your	journey	starts	with	one	step
Your	0.20	0.20	0.19	0.12	0.12	0.14
journey	0.13	0.23	0.23	0.12	0.10	0.15
starts	0.13	0.23	0.23	0.12	0.11	0.15
with	0.14	0.20	0.20	0.14	0.12	0.17
one	0.15	0.19	0.19	0.13	0.18	0.12
step	0.13	0.21	0.21	0.14	0.09	0.18

This row contains the attention weights (normalized attention scores) computed previously

The values in each row should add up to 1.0 or 100%.

Similar Steps Should Be Followed



Step1: Attention Scores Matrix

▶ # Step1: Attention Scores Matrix

```
attn_scores = torch.empty(6, 6)

for i, x_i in enumerate(inputs):
    for j, x_j in enumerate(inputs):
        attn_scores[i, j] = torch.dot(x_i, x_j)

print(attn_scores)
```

```
... tensor([[0.9995, 0.9544, 0.9422, 0.4753, 0.4576, 0.6310],
          [0.9544, 1.4950, 1.4754, 0.8434, 0.7070, 1.0865],
          [0.9422, 1.4754, 1.4570, 0.8296, 0.7154, 1.0605],
          [0.4753, 0.8434, 0.8296, 0.4937, 0.3474, 0.6565],
          [0.4576, 0.7070, 0.7154, 0.3474, 0.6654, 0.2935],
          [0.6310, 1.0865, 1.0605, 0.6565, 0.2935, 0.9450]])
```

▶ # We can achieve the same as above more efficiently via matrix multiplication:

```
attn_scores = inputs @ inputs.T
print(attn_scores)
```

```
... tensor([[0.9995, 0.9544, 0.9422, 0.4753, 0.4576, 0.6310],
          [0.9544, 1.4950, 1.4754, 0.8434, 0.7070, 1.0865],
          [0.9422, 1.4754, 1.4570, 0.8296, 0.7154, 1.0605],
          [0.4753, 0.8434, 0.8296, 0.4937, 0.3474, 0.6565],
          [0.4576, 0.7070, 0.7154, 0.3474, 0.6654, 0.2935],
          [0.6310, 1.0865, 1.0605, 0.6565, 0.2935, 0.9450]])
```

Step2: Attention Weights Matrix

```
▶ # Step2: Attention Weights Matrix

# we normalize each row so that the values in each row sum to 1
attn_weights = torch.softmax(attn_scores, dim=-1)
print(attn_weights)

... tensor([[0.2098, 0.2006, 0.1981, 0.1242, 0.1220, 0.1452],
            [0.1385, 0.2379, 0.2333, 0.1240, 0.1082, 0.1581],
            [0.1390, 0.2369, 0.2326, 0.1242, 0.1108, 0.1565],
            [0.1435, 0.2074, 0.2046, 0.1462, 0.1263, 0.1720],
            [0.1526, 0.1958, 0.1975, 0.1367, 0.1879, 0.1295],
            [0.1385, 0.2184, 0.2128, 0.1420, 0.0988, 0.1896]])
```

Verification

▶ # Quick verification that the values in each row indeed sum to 1:

```
row_2_sum = sum([0.1385, 0.2379, 0.2333, 0.1240, 0.1082, 0.1581])  
print("Row 2 sum:", row_2_sum)  
  
print("All row sums:", attn_weights.sum(dim=-1))
```

```
... Row 2 sum: 1.0  
All row sums: tensor([1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000])
```

step3: Compute All Context vectors

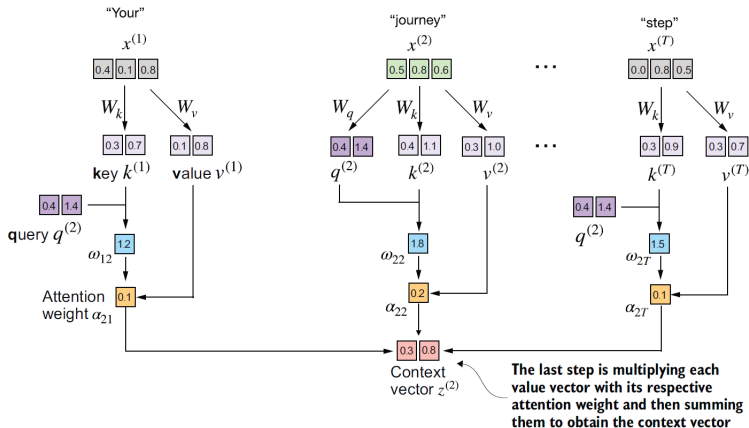
```
▶ all_context_vecs = attn_weights @ inputs
print(all_context_vecs)
# the previously computed context vector z(2)=[0.4419,0.6515,0.5683] can be found in the 2nd row:

... tensor([[0.4421, 0.5931, 0.5790],
            [0.4419, 0.6515, 0.5683],
            [0.4431, 0.6496, 0.5671],
            [0.4304, 0.6298, 0.5510],
            [0.4671, 0.5910, 0.5266],
            [0.4177, 0.6503, 0.5645]])
```

Current Section

- 1 Introduction
- 2 A simple self-attention mechanism without trainable weights
- 3 Implementing self-attention with trainable weights**
- 4 Causal Attention
- 5 Extending single-head attention to multi-head attention

How Final Step Looks Like



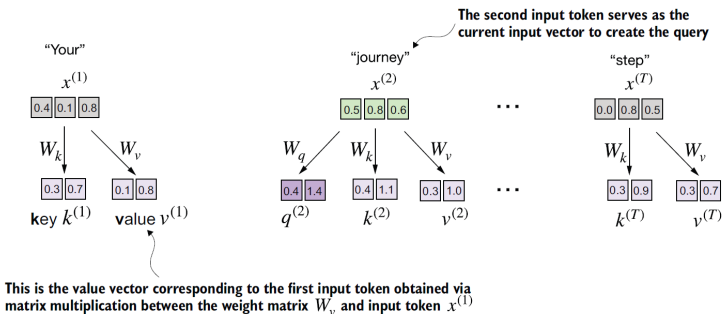
This is what we want to obtain at the end of this section: the context vector $z^{(2)}$. The size of the context vector can be controlled by choosing the dimension of the value vectors, which is determined by the value weight matrix W_v (This size is arbitrary and does not depend directly on the input embedding size, because it is determined by the trainable weight matrices).

Using Trainable Weight

We are implementing the self-attention mechanism that is used in the original transformer architecture, the GPT models, and most other popular LLMs.

The most notable difference is the introduction of weight matrices that are updated during model training. These trainable weight matrices are crucial so that the model (specifically, the attention module inside the model) can learn to produce "good" context vectors.

Step1: Computing Query (q), Key (k), and Value (v) Vectors for Input Elements x



In the first step of the self-attention mechanism with trainable weight matrices, we compute query (q), key (k), and value (v) vectors for input elements x . Similar to previous sections, we designate the second input, $x^{(2)}$, as the query input. The query vector $q^{(2)}$ is obtained via matrix multiplication between the input $x^{(2)}$ and the weight matrix W_q . Similarly, we obtain the key and value vectors via matrix multiplication involving the weight matrices W_k and W_v .

Self-Attention with Trainable Weight Matrices

- Implementing the self-attention mechanism step by step, we will start by introducing the three training weight matrices W_q , W_k , and W_v
- These three matrices are used to project the embedded input tokens, $x^{(i)}$, into query, key, and value vectors via matrix multiplication:
 - Query vector: $q^{(i)} = x^{(i)} W_q$
 - Key vector: $k^{(i)} = x^{(i)} W_k$
 - Value vector: $v^{(i)} = x^{(i)} W_v$
- The embedding dimensions of the input x and the query vector q can be the same or different, depending on the model's design and specific implementation
- In GPT models, the input and output dimensions are usually the same, but for illustration purposes, to better follow the computation, we choose different input and output dimensions here:

Code

```
▶ import torch

inputs = torch.tensor(
    [[0.43, 0.15, 0.89], # Your      (x^1)
     [0.55, 0.87, 0.66], # journey  (x^2)
     [0.57, 0.85, 0.64], # starts  (x^3)
     [0.22, 0.58, 0.33], # with    (x^4)
     [0.77, 0.25, 0.10], # one     (x^5)
     [0.05, 0.80, 0.55]] # step     (x^6)
)

x_2 = inputs[1] # second input element
d_in = inputs.shape[1] # the input embedding size, d=3
d_out = 2 # the output embedding size, d=2
```

Step1: Random Initialization

```
# we initialize the three weight matrices; note that we are setting requires_grad=False to reduce clutter in the outputs for illustration purposes,
# but if we were to use the weight matrices for model training, we would set requires_grad=True to update these matrices during model training
torch.manual_seed(123)
```

```
# torch.nn.Parameter makes them trainable
W_query = torch.nn.Parameter(torch.rand(d_in, d_out), requires_grad=False)
W_key   = torch.nn.Parameter(torch.rand(d_in, d_out), requires_grad=False)
W_value = torch.nn.Parameter(torch.rand(d_in, d_out), requires_grad=False)
```

W_query

```
Parameter containing:
tensor([[0.2961, 0.5166],
        [0.2517, 0.6886],
        [0.0740, 0.8665]])
```

W_key

```
Parameter containing:
tensor([[0.1366, 0.1025],
        [0.1841, 0.7264],
        [0.3153, 0.6871]])
```

W_value

```
Parameter containing:
tensor([[0.0756, 0.1966],
        [0.3164, 0.4017],
        [0.1186, 0.8274]])
```

Step1 Continued: Computing the Query, Key, and Value Vectors

```
query_2 = x_2 @ W_query # _2 because it's with respect to the 2nd input element
key_2 = x_2 @ W_key
value_2 = x_2 @ W_value
```

```
print(query_2)
```

```
tensor([0.4306, 1.4551])
```

▶ # we successfully projected the 6 input tokens from a 3D onto a 2D embedding space:

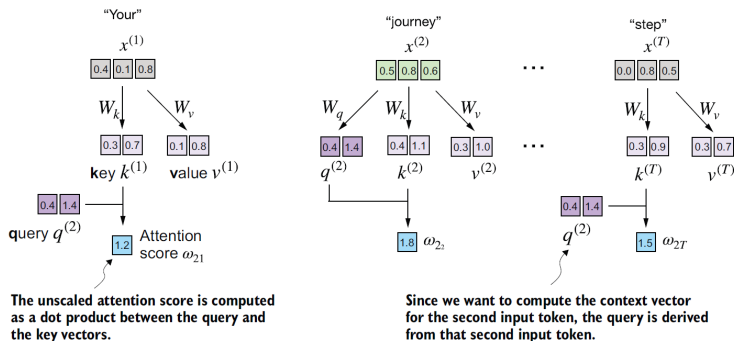
```
keys = inputs @ W_key
values = inputs @ W_value
```

```
print("keys.shape:", keys.shape)
print("values.shape:", values.shape)
```

```
... keys.shape: torch.Size([6, 2])
    values.shape: torch.Size([6, 2])
```

Step2: Compute the Unnormalized Attention Scores

We compute the unnormalized attention scores by computing the dot product between the query and each key vector:



The attention score computation is a dot-product computation similar to what we used in the simplified self-attention mechanism in previous section. The new aspect here is that we are not directly computing the dot product between the input elements, but using the query and key obtained by transforming the inputs via the respective weight matrices.

Code

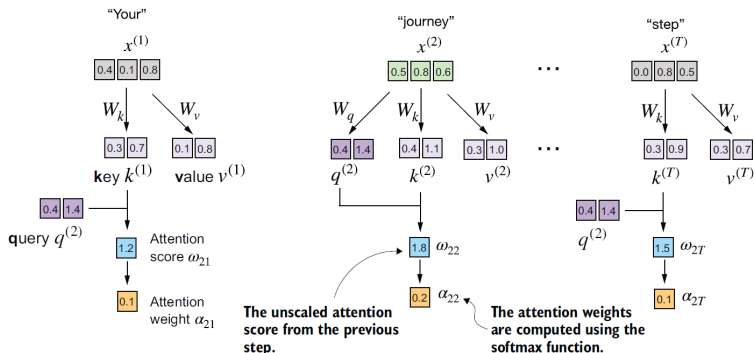
```
keys_2 = keys[1] # Python starts index at 0
attn_score_22 = query_2.dot(keys_2)
print(attn_score_22)
```

```
tensor(1.8524)
```

```
▶ # Since we have 6 inputs, we have 6 attention scores for the given query vector:
attn_scores_2 = query_2 @ keys.T # All attention scores for given query
print(attn_scores_2)
```

```
... tensor([1.2705, 1.8524, 1.8111, 1.0795, 0.5577, 1.5440])
```

Step3: Compute Attention Weights

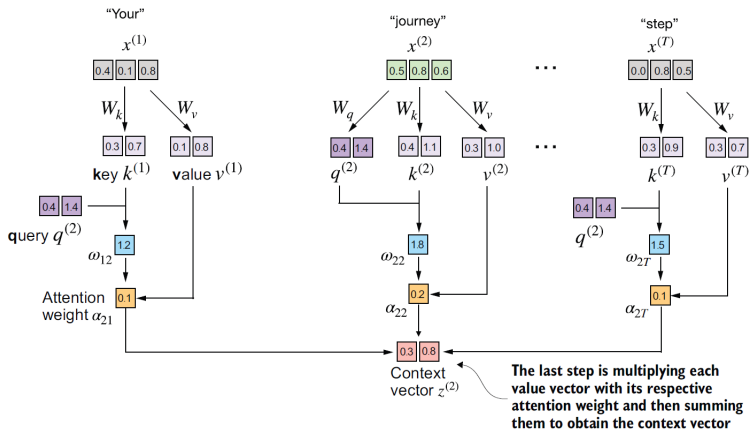


After computing the attention scores ω , the next step is to normalize these scores using the softmax function to obtain the attention weights α . We compute the attention weights (normalized attention scores that sum up to 1) using the softmax function we used earlier. The difference to earlier is that we now scale the attention scores by dividing them by the square root of the embedding dimension, $\sqrt{d_k}$: $\alpha = \text{softmax} \left(\frac{qK^T}{\sqrt{d_k}} \right)$

Code

```
▶ d_k = keys.shape[1]  
  attn_weights_2 = torch.softmax(attn_scores_2 / d_k**0.5, dim=-1)  
  print(attn_weights_2)  
  
... tensor([0.1500, 0.2264, 0.2199, 0.1311, 0.0906, 0.1820])
```

Step4: Compute the Context Vector



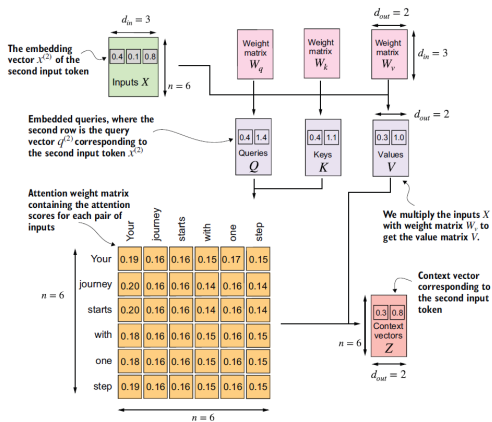
In the final step of the self-attention computation, we compute the context vector by combining all value vectors via the attention weights.

Code

```
▶ context_vec_2 = attn_weights_2 @ values  
  print(context_vec_2)  
  
... tensor([0.3061, 0.8210])
```

Generalization: Compute All Context Vectors

In self-attention, the input matrix X is transformed using three weight matrices, W_q , W_k , and W_v , to produce queries Q , keys K , and values V . The attention weights are computed from Q and K , then applied to V to produce the context vectors Z .



Implementing a compact SelfAttention class

```

▶ # Implementing a compact SelfAttention class
import torch.nn as nn

class SelfAttention_v1(nn.Module):

    def __init__(self, d_in, d_out):
        super().__init__()
        self.W_query = nn.Parameter(torch.rand(d_in, d_out))
        self.W_key = nn.Parameter(torch.rand(d_in, d_out))
        self.W_value = nn.Parameter(torch.rand(d_in, d_out))

    def forward(self, x):
        keys = x @ self.W_key
        queries = x @ self.W_query
        values = x @ self.W_value

        attn_scores = queries @ keys.T # omega
        attn_weights = torch.softmax(
            attn_scores / keys.shape[-1]**0.5, dim=-1
        )

        context_vec = attn_weights @ values
        return context_vec

d_in = inputs.shape[1] # the input embedding size, d=3
d_out = 2
torch.manual_seed(123)
sa_v1 = SelfAttention_v1(d_in, d_out)
print(sa_v1(inputs))

... tensor([[0.2996, 0.8053],
           [0.3061, 0.8210],
           [0.3058, 0.8203],
           [0.2948, 0.7939],
           [0.2927, 0.7891],
           [0.2990, 0.8040]], grad_fn=<MmBackward0>)
```

Using PyTorch's Linear Layers

We can simplify the implementation above by using PyTorch's `nn.Linear` layers. When the bias term is disabled, `nn.Linear` is equivalent to a matrix multiplication. Another advantage is that `nn.Linear` uses a better default weight initialization scheme than manually defining weights with `nn.Parameter(torch.rand(...))`, which helps make training more stable. Note that `SelfAttention_v1` and `SelfAttention_v2` give different outputs because they use different initial weights for the weight matrices

```
class SelfAttention_v2(nn.Module):

    def __init__(self, d_in, d_out, qkv_bias=False):
        super().__init__()
        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)

    def forward(self, x):
        keys = self.W_key(x)
        queries = self.W_query(x)
        values = self.W_value(x)

        attn_scores = queries @ keys.T
        attn_weights = torch.softmax(attn_scores / keys.shape[-1]**0.5, dim=-1)

        context_vec = attn_weights @ values
        return context_vec

torch.manual_seed(789)
sa_v2 = SelfAttention_v2(d_in, d_out)
print(sa_v2(inputs))
```

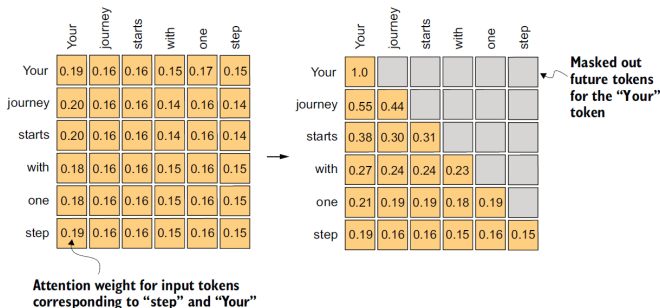
```
... tensor([[[-0.0739,  0.0713],
             [-0.0748,  0.0703],
             [-0.0749,  0.0702],
             [-0.0760,  0.0685],
             [-0.0763,  0.0679],
             [-0.0754,  0.0693]]], grad_fn=<MmBackward0>)
```

Current Section

- 1 Introduction
- 2 A simple self-attention mechanism without trainable weights
- 3 Implementing self-attention with trainable weights
- 4 Causal Attention**
- 5 Extending single-head attention to multi-head attention

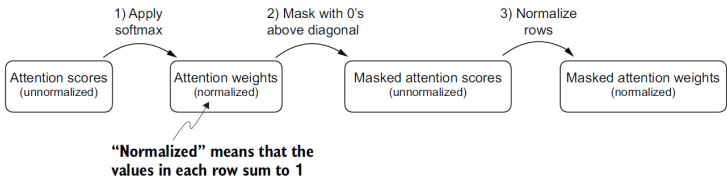
Causal Attention

In causal attention, the attention weights above the diagonal are masked, ensuring that for any given input, the LLM is unable to utilize future tokens while calculating the context vectors with the attention weight.



Applying a causal attention mask

One way to obtain the masked attention weight matrix in causal attention is to apply the softmax function to the attention scores, zeroing out the elements above the diagonal and normalizing the resulting matrix.



Code

Causal attention ensures that each next word prediction should only depend on the preceding words. To achieve this goal, for each given token, we mask out the future tokens.

```

▶ # Reuse the query and key weight matrices of the
  # SelfAttention_v2 object from the previous section for convenience
  queries = sa_v2.W_query(inputs)
  keys = sa_v2.W_key(inputs)
  attn_scores = queries @ keys.T

  attn_weights = torch.softmax(attn_scores / keys.shape[-1]**0.5, dim=-1)
  print(attn_weights)

... tensor([[0.1921, 0.1646, 0.1652, 0.1550, 0.1721, 0.1510],
           [0.2041, 0.1659, 0.1662, 0.1496, 0.1665, 0.1477],
           [0.2036, 0.1659, 0.1662, 0.1498, 0.1664, 0.1480],
           [0.1869, 0.1667, 0.1668, 0.1571, 0.1661, 0.1564],
           [0.1830, 0.1669, 0.1670, 0.1588, 0.1658, 0.1585],
           [0.1935, 0.1663, 0.1666, 0.1542, 0.1666, 0.1529]],
          grad_fn=<SoftmaxBackward0>)
```

Code

The simplest way to mask out future attention weights is by creating a mask via PyTorch's `tril` function with elements below the main diagonal (including the diagonal itself) set to 1 and above the main diagonal set to 0:

```
▶ context_length = attn_scores.shape[0]
  mask_simple = torch.tril(torch.ones(context_length, context_length))
  print(mask_simple)

... tensor([[1., 0., 0., 0., 0., 0.],
            [1., 1., 0., 0., 0., 0.],
            [1., 1., 1., 0., 0., 0.],
            [1., 1., 1., 1., 0., 0.],
            [1., 1., 1., 1., 1., 0.],
            [1., 1., 1., 1., 1., 1.]])
```

Code

Then, we can multiply the attention weights with this mask to zero out the attention scores above the diagonal:

```

▶ masked_simple = attn_weights*mask_simple
  print(masked_simple)

... tensor([[0.1921, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
            [0.2041, 0.1659, 0.0000, 0.0000, 0.0000, 0.0000],
            [0.2036, 0.1659, 0.1662, 0.0000, 0.0000, 0.0000],
            [0.1869, 0.1667, 0.1668, 0.1571, 0.0000, 0.0000],
            [0.1830, 0.1669, 0.1670, 0.1588, 0.1658, 0.0000],
            [0.1935, 0.1663, 0.1666, 0.1542, 0.1666, 0.1529]],
          grad_fn=<MulBackward0>)
```

However, if the mask were applied after softmax, like above, it would disrupt the probability distribution created by softmax. Softmax ensures that all output values sum to 1. Masking after softmax would require re-normalizing the outputs to sum to 1 again, which complicates the process and might lead to unintended effects.

Code—Rows Need to Sum to 1 for Optimization Purposes

To make sure that the rows sum to 1, we can normalize the attention weights as follows:

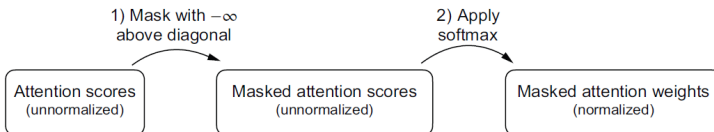
```

▶ row_sums = masked_simple.sum(dim=-1, keepdim=True)
  masked_simple_norm = masked_simple / row_sums
  print(masked_simple_norm)

... tensor([[1.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
           [0.5517, 0.4483, 0.0000, 0.0000, 0.0000, 0.0000],
           [0.3800, 0.3097, 0.3103, 0.0000, 0.0000, 0.0000],
           [0.2758, 0.2460, 0.2462, 0.2319, 0.0000, 0.0000],
           [0.2175, 0.1983, 0.1984, 0.1888, 0.1971, 0.0000],
           [0.1935, 0.1663, 0.1666, 0.1542, 0.1666, 0.1529]],
          grad_fn=<DivBackward0>)
```

More Efficient Way

A more efficient way to obtain the masked attention weight matrix in causal attention is to mask the attention scores with negative infinity values before applying the softmax function.



Code

While we are technically done with coding the causal attention mechanism now, let's briefly look at a more efficient approach to achieve the same as above. So, instead of zeroing out attention weights above the diagonal and renormalizing the results, we can mask the unnormalized attention scores above the diagonal with negative infinity before they enter the softmax function. As we can see below, now the attention weights in each row correctly sum to 1 again:

```
mask = torch.triu(torch.ones(context_length, context_length), diagonal=1)
masked = attn_scores.masked_fill(mask.bool(), -torch.inf)
print(masked)
```

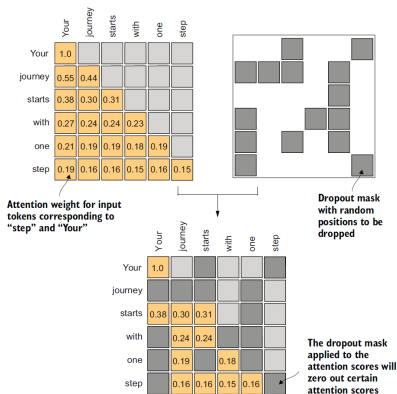
```
tensor([[0.2899, -inf, -inf, -inf, -inf, -inf],
        [0.4656, 0.1723, -inf, -inf, -inf, -inf],
        [0.4594, 0.1703, 0.1731, -inf, -inf, -inf],
        [0.2642, 0.1024, 0.1036, 0.0186, -inf, -inf],
        [0.2183, 0.0874, 0.0882, 0.0177, 0.0786, -inf],
        [0.3408, 0.1270, 0.1290, 0.0198, 0.1290, 0.0078]],
      grad_fn=<MaskedFillBackward0>)
```

```
▶ attn_weights = torch.softmax(masked / keys.shape[-1]**0.5, dim=-1)
print(attn_weights)
```

```
... tensor([[1.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
           [0.5517, 0.4483, 0.0000, 0.0000, 0.0000, 0.0000],
           [0.3800, 0.3097, 0.3103, 0.0000, 0.0000, 0.0000],
           [0.2758, 0.2460, 0.2462, 0.2319, 0.0000, 0.0000],
           [0.2175, 0.1983, 0.1984, 0.1888, 0.1971, 0.0000],
           [0.1935, 0.1663, 0.1666, 0.1542, 0.1666, 0.1529]],
      grad_fn=<SoftmaxBackward0>)
```

Masking additional attention weights with dropout

In addition, we also apply dropout to reduce overfitting during training. Dropout can be applied in several places: for example, after computing the attention weights; or after multiplying the attention weights with the value vectors. Here, we will apply the dropout mask after computing the attention weights because it's more common. Furthermore, in this specific example, we use a dropout rate of 50%, which means randomly masking out half of the attention weights. (When we train the GPT model later, we will use a lower dropout rate, such as 0.1 or 0.2).



Code

If we apply a dropout rate of 0.5 (50%), the non-dropped values will be scaled accordingly by a factor of $1/0.5 = 2$. The scaling is calculated by the formula $1 / (1 - \text{dropout_rate})$.

```

▶ torch.manual_seed(123)
  dropout = torch.nn.Dropout(0.5) # dropout rate of 50%
  example = torch.ones(6, 6) # create a matrix of ones

  print(dropout(example))

... tensor([[2., 2., 0., 2., 2., 0.],
           [0., 0., 0., 2., 0., 2.],
           [2., 2., 2., 2., 0., 2.],
           [0., 2., 2., 0., 0., 2.],
           [0., 2., 0., 2., 0., 2.],
           [0., 2., 2., 2., 2., 0.]])

```

```

▶ torch.manual_seed(123)
  print(dropout(attn_weights))

... tensor([[2.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
           [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
           [0.7599, 0.6194, 0.6206, 0.0000, 0.0000, 0.0000],
           [0.0000, 0.4921, 0.4925, 0.0000, 0.0000, 0.0000],
           [0.0000, 0.3966, 0.0000, 0.3775, 0.0000, 0.0000],
           [0.0000, 0.3327, 0.3331, 0.3084, 0.3331, 0.0000]],
          grad_fn=<MulBackward0>)

```

Dropout scales each remaining activation by $\frac{1}{1-p}$ so that each activation keeps the same expected value after some values are randomly set to zero.

Implementing a compact causal self-attention class

Now, we are ready to implement a working implementation of self-attention, including the causal and dropout masks. One more thing is to implement the code to handle batches consisting of more than one input so that our CausalAttention class supports the batch outputs produced by the data loader we implemented in previous module. For simplicity, to simulate such batch input, we duplicate the input text example:

```
▶ batch = torch.stack((inputs, inputs), dim=0)
  print(batch.shape) # 2 inputs with 6 tokens each, and each token has embedding dimension 3
... torch.Size([2, 6, 3])
```

Code

```
class CausalAttention(nn.Module):

    def __init__(self, d_in, d_out, context_length,
                  dropout, qkv_bias=False):
        super().__init__()
        self.d_out = d_out
        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.dropout = nn.Dropout(dropout) # New
        self.register_buffer('mask', torch.triu(torch.ones(context_length, context_length), diagonal=1)) # New

    def forward(self, x):
        b, num_tokens, d_in = x.shape # New batch dimension b
        # For inputs where `num_tokens` exceeds `context_length`, this will result in errors
        # in the mask creation further below.
        # In practice, this is not a problem since the LLM (chapters 4-7) ensures that inputs
        # do not exceed `context_length` before reaching this forward method.
        keys = self.W_key(x)
        queries = self.W_query(x)
        values = self.W_value(x)

        attn_scores = queries @ keys.transpose(1, 2) # Changed transpose
        attn_scores.masked_fill_( # New, _ ops are in-place
            self.mask.bool()[:num_tokens, :num_tokens], -torch.inf)
        # `num_tokens` to account for cases where the number of tokens in the batch is smaller than the supported context_size
        attn_weights = torch.softmax(
            attn_scores / keys.shape[-1]**0.5, dim=-1
        )
        attn_weights = self.dropout(attn_weights) # New

        context_vec = attn_weights @ values
        return context_vec
```

Code

```
torch.manual_seed(123)

context_length = batch.shape[1]
ca = CausalAttention(d_in, d_out, context_length, 0.0)

context_vecs = ca(batch)

print(context_vecs)
print("context_vecs.shape:", context_vecs.shape)

tensor([[[[-0.4519,  0.2216],
          [-0.5874,  0.0058],
          [-0.6300, -0.0632],
          [-0.5675, -0.0843],
          [-0.5526, -0.0981],
          [-0.5299, -0.1081]],
         [[[-0.4519,  0.2216],
          [-0.5874,  0.0058],
          [-0.6300, -0.0632],
          [-0.5675, -0.0843],
          [-0.5526, -0.0981],
          [-0.5299, -0.1081]]], grad_fn=<UnsafeViewBackward0>])
context_vecs.shape: torch.Size([2, 6, 2])
```

Current Section

- 1 Introduction
- 2 A simple self-attention mechanism without trainable weights
- 3 Implementing self-attention with trainable weights
- 4 Causal Attention
- 5 Extending single-head attention to multi-head attention**

Multi-head Attention

We simply stack multiple single-head attention modules to obtain a multi-head attention module:

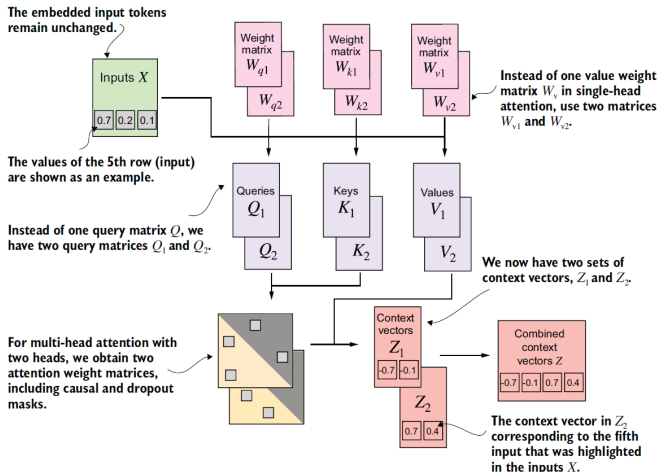
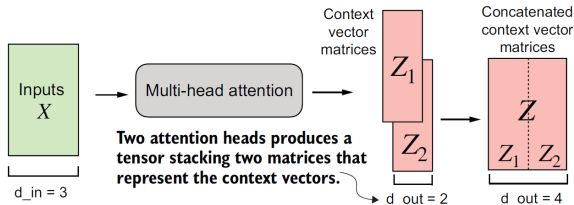


Figure Explanation

The multi-head attention module contains multiple single-head attention modules running in parallel. In previous slide's Figure, instead of using one value weight matrix W_v , a two-head attention module uses two value weight matrices, W_{v1} and W_{v2} . The same applies to the query and key weight matrices, W_q and W_k . This produces two sets of context vectors, Z_1 and Z_2 , which are combined into a single context matrix Z .

Main Idea



Choosing an embedding dimension of 2 ($d_{out} = 2$) for the context vectors results in a final embedding dimension of 4 ($d_{out} \times \text{num_heads}$).

The main idea behind multi-head attention is to run the attention mechanism multiple times (in parallel) with different, learned linear projections. This allows the model to jointly attend to information from different representation subspaces at different positions.

Code

```

class MultiHeadAttentionWrapper(nn.Module):

    def __init__(self, d_in, d_out, context_length, dropout, num_heads, qkv_bias=False):
        super().__init__()
        self.heads = nn.ModuleList(
            [CausalAttention(d_in, d_out, context_length, dropout, qkv_bias)
             for _ in range(num_heads)]
        )

    def forward(self, x):
        return torch.cat([head(x) for head in self.heads], dim=-1)

torch.manual_seed(123)

context_length = batch.shape[1] # This is the number of tokens
d_in, d_out = 3, 2
mha = MultiHeadAttentionWrapper(
    d_in, d_out, context_length, 0.0, num_heads=2
)

context_vecs = mha(batch)

print(context_vecs)
print("context_vecs.shape:", context_vecs.shape)

... tensor([[[[-0.4519,  0.2216,  0.4772,  0.1063],
               [-0.5874,  0.0058,  0.5891,  0.3257],
               [-0.6300, -0.0632,  0.6202,  0.3860],
               [-0.5675, -0.0843,  0.5478,  0.3589],
               [-0.5526, -0.0981,  0.5321,  0.3428],
               [-0.5299, -0.1081,  0.5077,  0.3493]],
              [[[-0.4519,  0.2216,  0.4772,  0.1063],
               [-0.5874,  0.0058,  0.5891,  0.3257],
               [-0.6300, -0.0632,  0.6202,  0.3860],
               [-0.5675, -0.0843,  0.5478,  0.3589],
               [-0.5526, -0.0981,  0.5321,  0.3428],
               [-0.5299, -0.1081,  0.5077,  0.3493]]], grad_fn=<CatBackward0>])
context_vecs.shape: torch.Size([2, 6, 4])

```

Implementing multi-head attention with weight splits

While the implementation above is intuitive and fully functional, we can also write a stand-alone `MultiHeadAttention` class to achieve the same result. Instead of explicitly concatenating separate attention heads, we create single weight matrices, W_q , W_k , and W_v , and then split their outputs into separate heads.

Code

```
#Implementing multi-head attention with weight splits
class MultiHeadAttention(nn.Module):
    def __init__(self, d_in, d_out, context_length, dropout, num_heads, qkv_bias=False):
        super().__init__()
        assert (d_out % num_heads == 0), \
            "d_out must be divisible by num_heads"

        self.d_out = d_out
        self.num_heads = num_heads
        self.head_dim = d_out // num_heads # Reduce the projection dim to match desired output dim

        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.out_proj = nn.Linear(d_out, d_out) # Linear layer to combine head outputs
        self.dropout = nn.Dropout(dropout)
        self.register_buffer(
            "mask",
            torch.triu(torch.ones(context_length, context_length),
                        diagonal=1)
        )
```

Code—Continued

```
def forward(self, x):
    b, num_tokens, d_in = x.shape
    # As in 'CausalAttention', for inputs where 'num_tokens' exceeds 'context_length',
    # this will result in errors in the mask creation further below.
    # In practice, this is not a problem since the LLM (chapters 4-7) ensures that inputs
    # do not exceed 'context_length' before reaching this forward

    keys = self.W_key(x) # Shape: (b, num_tokens, d_out)
    queries = self.W_query(x)
    values = self.W_value(x)

    # We implicitly split the matrix by adding a 'num_heads' dimension
    # Unroll last dim: (b, num_tokens, d_out) -> (b, num_tokens, num_heads, head_dim)
    keys = keys.view(b, num_tokens, self.num_heads, self.head_dim)
    values = values.view(b, num_tokens, self.num_heads, self.head_dim)
    queries = queries.view(b, num_tokens, self.num_heads, self.head_dim)

    # Transpose: (b, num_tokens, num_heads, head_dim) -> (b, num_heads, num_tokens, head_dim)
    keys = keys.transpose(1, 2)
    queries = queries.transpose(1, 2)
    values = values.transpose(1, 2)

    # Compute scaled dot-product attention (aka self-attention) with a causal mask
    attn_scores = queries @ keys.transpose(2, 3) # Dot product for each head

    # Original mask truncated to the number of tokens and converted to boolean
    mask_bool = self.mask.bool()[:num_tokens, :num_tokens]

    # Use the mask to fill attention scores
    attn_scores.masked_fill_(mask_bool, -torch.inf)

    attn_weights = torch.softmax(attn_scores / keys.shape[-1]**0.5, dim=-1)
    attn_weights = self.dropout(attn_weights)

    # Shape: (b, num_tokens, num_heads, head_dim)
    context_vec = (attn_weights @ values).transpose(1, 2)

    # Combine heads, where self.d_out = self.num_heads * self.head_dim
    context_vec = context_vec.contiguous().view(b, num_tokens, self.d_out)
    context_vec = self.out_proj(context_vec) # optional projection

    return context_vec
```

Code—Continued

```

torch.manual_seed(123)

batch_size, context_length, d_in = batch.shape
d_out = 2
mha = MultiHeadAttention(d_in, d_out, context_length, 0.0, num_heads=2)

context_vecs = mha(batch)

print(context_vecs)
print("context_vecs.shape:", context_vecs.shape)

```

```

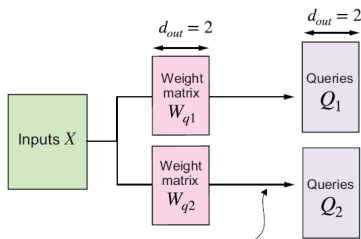
... tensor([[[[0.3190, 0.4858],
               [0.2943, 0.3897],
               [0.2856, 0.3593],
               [0.2693, 0.3873],
               [0.2639, 0.3928],
               [0.2575, 0.4028]],

              [[0.3190, 0.4858],
               [0.2943, 0.3897],
               [0.2856, 0.3593],
               [0.2693, 0.3873],
               [0.2639, 0.3928],
               [0.2575, 0.4028]]], grad_fn=<ViewBackward0>)]
context_vecs.shape: torch.Size([2, 6, 2])

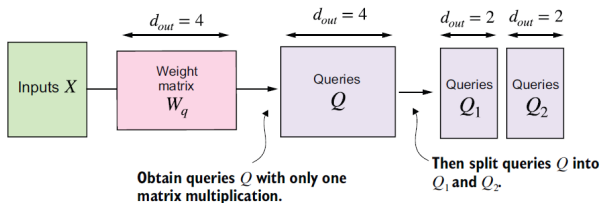
```

The implementation above is a more efficient version of `MultiHeadAttentionWrapper`. The output may look slightly different because the random weight initializations are different, but both implementations are fully functional and can be used in the GPT model. We also add an output projection layer, `self.out_proj`, which is a linear transformation that keeps the same dimensions. This projection layer is commonly used in LLMs, although it is not strictly required.

Visualization



Perform two matrix multiplications to obtain the two query matrices, Q_1 and Q_2 .



Congratulations!

Now you understand how the attention mechanism works in an LLM!

